I. forduló (Hagyományos verseny) 2026. november 6-8.

Egy régi, elhagyott bánya mélyén labirintusként kanyarognak a kamrák. A bányászok egykor sietve hagyták el őket, és a sötétben ma is csillogó aranyrögök várják, hogy valaki rájuk leljen. A bánya azonban nem kegyelmez, és egyetlen rossz lépés után már a fal állja az utat. A kincseket csak egy acélból kovácsolt felfedező, a kincskereső robot szerezheti meg. Sorról sorra hajtja végre a neki írt parancsokat: a sötétben csak azt érzékeli, ami közvetlenül előtte van, így tapogatózva halad előre, és ahol aranyra bukkan, ott kibányássza a rögöt. De a robot csak annyit tud, amennyit a programja elárul neki, és egyetlen elrontott utasítás elég ahhoz, hogy falnak menjen.

A feladatotok egy olyan fejlesztői környezet (IDE) elkészítése, amelyben egy speciális programnyelven (MineScriptben) megírt program alapján vezérelhető a robot a megadott labirintusban. A fejlesztői környezetnek lehetőséget kell biztosítania a program átszerkesztésére, futtatására és akár soronkénti nyomkövetésére is, valamint a futás lépéseinek naplózására egy kimeneti fájlba. A környezet részét képezi egy robot szimulátor is, amely a labirintusban megjeleníti a robot mozgását, az általa felfedezett területet és a megtalált aranyat, és segítségével a robot manuálisan is irányítható.

1. Az alkalmazás bemenete

Az alkalmazás indulásakor két paramétert vár. Első paraméterként az input fájl nevét kell megadni, második paraméterként pedig a naplófájl nevét.

Például, ha az alkalmazásotok lefordított, futtatható fájljának neve `robot.exe`, akkor a:

```
robot.exe input_001.txt output_001.txt
```

utasítás hatására az alkalmazásotok olvassa be az `input_001.txt` állományt, majd ezután kezdje el az igazi működését a fejlesztői környezet. A naplófájl neve `output_001.txt` lesz, aminek pontos létrehozásáról a naplózásról szóló fejezetben lesz szó.

Az alkalmazás ún. csendes üzemmódban is elindítható három paraméter megadásával az alábbi módon:

```
robot.exe -q input_001.txt output_001.txt
```

Első paraméterként tehát egy plusz kapcsoló (`-q`) kerül megadásra. Ilyenkor a programotoknak automatikusan - külön felhasználói interakció nélkül - fel kell dolgoznia a megadott bemeneti állományt és közben a naplófájlt ki kell írnia. A naplófájl kiírása után - felhasználói interakció nélkül - a program automatikusan kilép.

Az input fájl tartalmaz egy térképet és utána egy MineScriptben megírt programkódot, ez irányítja a robotot.

Példa:

```
3 15
....@..........
.##############.
.......@...>...
set arany to isThere 2
print arany
```

A fájl első sorában a térkép kiterjedése található, egy szóközzel van elválasztva a két dimenzió. (Mindkét dimenzió legalább 1, maximum 15.) A példában 3 sorból és 15 oszlopból áll a térkép. A térképen szereplő szimbólumok jelentése:

- a . üres kamra, amiben épp nem található se a robot, se aranyrög,
- a `@` olyan kamra, amelyben aranyrög található,
- a `#` a fal, ami áthatolhatatlan kőzet,
- míg a `>`, `<`, `^`, `v` szimbólumok a robotot jelölik (a térképen pontosan egy robot van). A robot egy aranyrögöt nem tartalmazó kamrában áll, a szimbólum pedig a robot irányát is megadja, vagyis azt, hogy a négy égtáj közül merre néz: `>` kelet, `<` nyugat, `^` észak, `v` dél. A példában szereplő `>` tehát azt jelenti, hogy a robot épp kelet felé néz.

A térkép után azonnal következnek a robotot irányító program utasításai. (A program lehet üres is.)

A példaprogram működése:

- Az `isThere x` függvény hívása logikai értékkel (`true`/`false`) tér vissza aszerint, hogy a robot előtti mezőn az `x` paraméternek megfelelő objektum található-e: `0` esetén üres kamra, `1` esetén fal, `2` esetén aranyrög.
- az `isThere 2` tehát megvizsgálja, hogy a robot előtti mezőn aranyrög van-e,
- ennek az eredményét (logikai érték) az `arany` nevű változóba helyezi el,
- majd kiírja a konzolra az `arany` változó értékét.

A bemeneti fájl formátuma mindig helyes, így nem kell arra készülnötök, hogy formai hiba lenne benne.

2. A szöveges forráskód nyelvtana

Tekintsük az alábbi programot, ami addig vezeti a robotot előre, amíg az falba nem ütközik. Útközben a robot minden aranyrögöt kibányász, majd a program a futása végén kiírja, mennyi aranyrögöt bányászott ki.

```
set count to 0
set fal to isThere 1
set megy to not fal
while megy
  set arany to isThere 2
  if arany
    step forward 1
    mine here
    set count to count + 1
  else
    step forward 1
  set fal to isThere 1
  set megy to not fal
print count
```

Az EBNF (Extended Backus-Naur Form) egy formális jelölésrendszer, amellyel egy nyelv nyelvtana – vagyis az érvényes szerkezetek felépítése – tömören leírható: megadja, hogy az egyes elemek (tokenek, azaz a nyelv legkisebb önálló egységei, pl. kulcsszavak, azonosítók, számok) milyen sorrendben és milyen ismétlődéssel állhatnak össze.

Egy MineScript nyelven írt program a következő EBNF-jelöléssel definiálható:

```
programme     = { command }

command       = procedure | set
              | if | ifelse | while

procedure     = step | turn | mine | print
function      = isThere

step          = "step" "forward" simpleval
turn          = "turn" "by" simpleval
mine          = "mine" "here"
print         = "print" simpleval

isThere       = "isThere" simpleval

set           = "set" ident "to" calculatedval
if            = "if" simpleval block
ifelse        = "if" simpleval block "else" block
while         = "while" simpleval block

block         = { command }

calculatedval = simpleval
              | function
              | binop
              | unop

binop         = simpleval binoperator simpleval
unop          = unoperator simpleval

simpleval     = ident | number | boolean-lit
```

```
binoperator   = "+" | "-" | "*"
              | "=" | "<>" | "<" | "<=" | ">" | ">="
              | "and" | "or"
unoperator    = "not"

boolean-lit   = "true" | "false"
ident         = letter { letter | digit }
number        = digit { digit }

letter        = "a".."z" | "A".."Z"
digit         = "0".."9"
```

A kulcsszavak és az azonosítók is kis- és nagybetűérzékenyek: a kulcsszavakat pontosan a nyelvtanban megadott alakban kell írni (pl. `isThere`, nem `isthere`), az `arany` és az `Arany` pedig két különböző változó. A kódban szereplő egész számok 0 és 100 közöttiek lehetnek.

Az azonosítók (`ident`) nem lehetnek kulcsszavak, azaz az alábbi szavak egyike sem (semmilyen kis- vagy nagybetűs kombinációban): `and`, `by`, `else`, `false`, `forward`, `here`, `if`, `isThere`, `mine`, `not`, `or`, `print`, `set`, `step`, `to`, `true`, `turn`, `while`.

2.1 Soronkénti feldolgozás / tokenizálás

A forráskód tehát tokenekből épül fel, ezek a program legkisebb, önálló jelentéssel bíró egységei: a kulcsszavak (pl. `set`), a logikai értékek (pl. `true`), az operátorok (pl. `+`), az azonosítók (változónevek) és a számok. Amennyiben egy szöveges állományból szeretnénk feldolgozni a forráskódot, az EBNF-jelölés önmagában még nem elegendő, mert először a szövegben kell tudni helyesen beazonosítani a tokeneket. Szerencsére ez a folyamat nem bonyolult, és az alábbi algoritmus mentén elvégezhető:

1. **Sorokra bontás.** Minden sor egy utasítás, nincs külön pontosvessző vagy más elválasztó. A sor eleji behúzás adja a szintet, aminek egy szinten belül egységesnek kell lennie. (Ennek egyezőségét nem kell ellenőrizni, feltételezhetitek, hogy a beolvasott forráskódban így szerepel.) A behúzás jelöli, mi tartozik egy `if`/`while` törzséhez; amint csökken, a törzs lezárul. Az `else` az `if` törzsével azonos behúzáson van. A behúzás csak szóközzel jelölhető (tab nem használható). A különböző blokkok esetén más és más lehet a behúzás mértéke (de a blokkon belül egységes).
2. **Tokenizálás.** A behúzás után a sort szóközök mentén daraboljuk tokenekre. Pontosan egy szóköz szerepel az egy soron belüli tokenek között, kivéve a sor elejét; ott - az előző pontban leírtak szerint - a behúzásnak megfelelően vannak a szóközök.
3. **A sor értelmezése.** Az első token adja meg, milyen utasításról van szó (`step`, `turn`, `mine`, `print`, `set`, `if`, `else`, `while`). Az utasítás kötelező kulcsszavai rögzítettek (`step forward …`, `turn by …`, `mine here`, `while …`, `set … to …`), a nem rögzített tokenek pedig `simpleval`-ok.
4. Nincs beágyazott kifejezés, nincs rekurzió: minden `simpleval` egyetlen token. Ezért elég a sorolvasás és a szódarabolás.
5. A forráskód nem tartalmaz utasítás nélküli - azaz üres - sorokat. (És ezt nem szükséges ellenőrizni sem.)

Feltételezhetitek, hogy a beolvasandó forráskód ezen algoritmus alapján hiba nélkül tokenizálható.

2.2 Szemantikai szabályok

Míg a nyelvtan (EBNF) csak a program struktúráját írja le, addig az ún. szemantikai szabályok a jelentését és a végrehajtás közbeni elvárásokat rögzítik. Azaz a szemantikai szabályok azt határozzák meg, hogy a szintaktikailag helyes program hogyan viselkedik futás közben: mit jelentenek az egyes utasítások, milyen típusokkal dolgozhatunk, és mikor számít egy program hibásnak.

Számos szemantikai szabály tartozik a MineScript nyelvhez, amelyek rendeltetésük szerint csoportosíthatóak.

2.2.1 Kulcsszavak működése

- **Aritmetika** (`+ - *`): egész operandusok, egész eredmény.
- **Összehasonlítás** (`< <= > >=`): egész operandusok, logikai eredmény.
- **Egyenlőségvizsgálat** (`= <>`): egész és logikai operandusokra is működik, de mindkét operandusnak azonos típusúnak kell lennie (mindkettő egész vagy mindkettő logikai). Az eredmény logikai.
- **Logika** (`and`, `or`, `not`): logikai operandusok, logikai eredmény.
- `set`: a `set x to ertek` utasítás először megpróbálja kiértékelni a `to` utáni kifejezést, majd siker esetén az eredményt az `x` nevű változóba írja. Ha az `x` még nem létezik, az első értékadásnál jön létre, és az érték típusa lesz a változó végleges típusa. Ha már létezik, az új értéknek azonos típusúnak kell lennie a korábbival. A `to` utáni rész (`calculatedval`) kiértékelése balról jobbra történik: ha művelet áll benne, annak operandusait egyenként olvassa be, és mindegyiket azonnal ellenőrzi, hogy a típusa megfelel-e a műveletnek, mielőtt a következőre lépne. Az első hibánál a kiértékelés megáll. Ha az eredmény már az első operandusból következne (pl. `set x = true or c`), a másodikat akkor is beolvassa. Az = és a <> első operandusa bármilyen típusú lehet, náluk a két típus egyezését a második operandus beolvasása után ellenőrzi.
- `if`**:** a feltétel szerint a `then`- vagy (ha van) az `else`-ág fut.
- `while`**:** a törzs addig ismétlődik, amíg a logikai feltétel igaz.

2.2.2 A nyelv alapszabályai

- **Két típus:** egész (integer) és logikai (boolean), köztük nincs implicit konverzió.
- **Nincs változódeklaráció:** a változó az első `set`-kor jön létre, és a program végrehajtásának végéig él.
- **Típusrögzítés:** a változó típusa az első értékadáskor véglegesen rögzül, később nem változtatható.
- **Feltétel:** az `if` és a `while` feltétele logikai kell legyen;
- **Minden típusellenőrzés futásidőben** történik, nincs fordítás előtti ellenőrzés.

- **Végrehajtás:** a sorok fentről lefelé, szekvenciálisan futnak; a blokk az azonos mélységbe behúzott sorokból áll.

2.2.3 Robot API

A robot vezérlése a robot API meghívásán keresztül lehetséges. Az egyes API-végpontok natívan a MineScript nyelv részét alkotják.

- `step forward n`: a robot egyetlen hívással `n` egységet halad előre (`n` nemnegatív egész), közben a köztes mezőket is érintve. Ha olyan mezőt érint, ahol aranyrög található, ott hagyja.
- `turn by n`: az `n` értéke nemnegatív. A robot `n` negyedfordulatot lép balra (azaz az óramutató járásával ellentétes irányba). `0` esetén a robot iránya nem változik.
- `mine here`: a robot a mezőn, amelyen áll, kibányássza az aranyrögöt, ezután már ezen a mezőn nem lesz több bányászható arany;
- `isThere w`: a robot előtt lévő mező fajtáját ellenőrzi: a `w` paraméter (`0` = üres kamra, `1` = fal, `2` = aranyrög) alapján logikai értékkel tér vissza (`true`/`false`). Amennyiben a robot vizsgálódása a térkép szélét meghaladja, azt falnak tekinti és ekképpen tér vissza a megfelelő `isThere` hívásra.
- `print x`: kiírja az egész vagy logikai értéket a konzolra.

(Az `n`, `w` és `x` paraméter - a nyelvtan alapján - mind `simpleval` kifejezés.)

2.2.4 Futásidejű hibák

A program **végrehajtása során** az alábbi hibák következhetnek be, amiket az alkalmazásotoknak kezelnie szükséges:

- **Típust váltó értékadás (**hibakód**:** `E_TYPE_MISMATCH`): egy már rögzített típusú változóhoz eltérő típusú értéket rendelsz.
- **Nem-inicializált változó olvasása (**hibakód**:** `E_UNINITIALIZED`): érték hozzárendelése előtt hivatkozol egy változóra.
- **Nem-logikai feltétel (**hibakód**:** `E_NON_BOOL_CONDITION`): az `if` vagy `while` feltétele nem logikai (pl. egész).
- **Rossz operandustípus (**hibakód**:** `E_BAD_OPERAND`): aritmetikai (`+ - *`) vagy összehasonlító (`< <= > >=`) művelet nem egész operandussal, logikai művelet (`and, or, not`) nem logikai operandussal, illetve egyenlőségvizsgálat (`= <>`) két különböző típusú operandussal.
- **Falra lépés vagy lelépés a térképről (**hibakód**:** `E_BEDROCK_HIT`): `step forward` a soron következő lépésnél falra vezetne vagy lelépne a térképről.
- **Bányászás arany nélkül (**hibakód**:** `E_NO_GOLD`): `mine here` olyan mezőn, ahol nincs vagy nincs már arany.
- **Érvénytelen paraméter (hibakód:** `E_BAD_ARG`**)**: egy API-hívás a megengedett tartományon kívüli paraméterrel vagy rossz típusú paraméterrel hívódik meg. Pl. `isThere w`, ahol `w` nem a 0, 1 vagy 2 értékek egyike, vagy `turn by true`.

Ezen hibák előfordulása esetén a robotprogram végrehajtása befejeződik.

3. A fejlesztői környezet működése

A megvalósítandó alkalmazás tehát egy fejlesztői környezet (Integrated Development Environment, azaz IDE), amelyben a következő funkciók érhetőek el:

- az input fájl betöltése a térképpel és a programkóddal,
- a program megtekintése szöveges forráskód és vizuális blokk-kód nézetben,
- a program szerkesztése vizuális blokkszerkesztővel (szerkeszteni csak így kell tudni a programot),
- a program futtatása nyomkövetés nélkül lassítva, hogy az jól követhető legyen,
- a program futtatása nyomkövetéssel: lépésről lépésre, a változók aktuális értékének megmutatásával, a nyomkövetésnek a szöveges forráskód vagy a vizuális blokk-kód nézet közül csak az egyikben kell működnie (a két megoldás egyenértékű),
- a program utolsó futásának naplózása a program indításakor megadott kimeneti fájlba.

Amennyiben elkezdődik a program futtatása, a térkép és a robot helyzete alaphelyzetbe kerül.

3.1 A vizuális blokk-kód szerkesztése

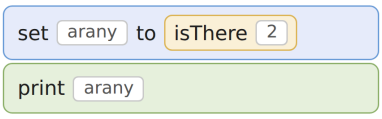
A program kódját - a szöveges forráskód mellett - ún. vizuális blokk-kóddal is ábrázolhatjuk. A vizuális blokk-kód és annak szöveges reprezentációja között kölcsönösen egyértelmű a megfeleltetés, azaz minden vizuális blokk-kódhoz pontosan egy szöveges reprezentáció tartozik és minden szöveges forráskódhoz is pontosan egy vizuális blokk-kód tartozik.

A vizuális blokk-kód a programozással ismerkedő újoncoknak segíti a programozás elsajátítását, mert egyrészt vizuális, másrészt pedig csak olyan részek illeszthetőek egymásba, amik struktúrájukban egy program érvényes szerkezetét adhatják.

Tekintsük a korábbi egyszerű szöveges forráskódot:

```
set arany to isThere 2
print arany
```

Ez az egyszerű program egy ilyen vizuális blokk-kóddal rajzolható le:



Látható, hogy a két programsor utasítása, egy-egy blokkot képez, egymás alatt van a két blokk. A fehér téglalapban lévő szerkeszthető részek lehetőséget biztosítanak arra, hogy változóneveket és számokat vagy logikai értéket adjunk meg, ezáltal pontosan definiálhatjuk az adott utasítást. A program tehát blokkokból, azok egymás alá illesztésével épül fel.

A felületnek egy munkaterületet szükséges biztosítani, amelyen a már összeállított, félkész programtörzs látható: a blokkok egymás után, a vezérlőblokkok törzsében pedig beljebb rendezve. A programhoz új blokkot menüből lehet választani:

- parancsblokkok (`step forward`, `turn by`, `mine here`, `print`, `set`)
- vezérlőblokkok: `if`, `if-else`, `while`
- érték-blokkok: a `set … to` ... parancsblokk `to` utáni részére illeszthető aritmetikai, összehasonlító és logikai műveletek

A kiválasztott blokkot a program egy megadott pozíciójába lehet beszúrni: parancsblokk másik blokk elé vagy mögé, illetve egy vezérlőblokk törzsébe.

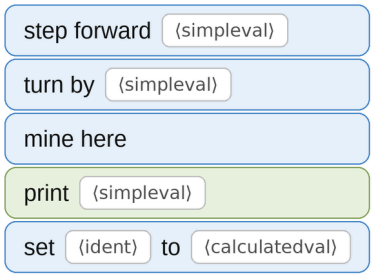
Hogy mindez grafikusan egérrel, vagy szöveges/konzolos felületen, menüpontok és pozíciók megadásával valósul-e meg, rátok van bízva — a követelmény a blokk-alapú, szerkezetileg mindig érvényes szerkesztés, nem egy konkrét megjelenítés.

A szerkesztésnek kényelmesnek kell lennie, ezért ne feltételezzétek, hogy a felhasználó lineárisan, fentről lefelé építi fel a programot. Előfordulhat például, hogy valaki először pl. egy `while x` blokkot hoz létre, és a `set x to ...` utasítást csak később helyezi el elé. Ezt a szerkesztőnek engednie kell, mert a program a szerkesztés közben még nem feltétlenül hibamentes, csak a végére kell azzá válnia (de ez majd a program futtatásakor fog eldőlni). Arra viszont ügyeljetek, hogy olyan változónevet ne lehessen létrehozni, amely a fent leírt nyelvtani szabályok szerint kulcsszónak minősül.

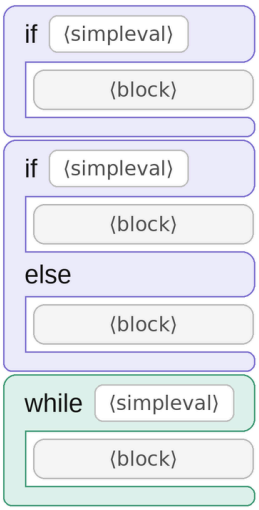
3.2 A blokkok

A vizuális blokk-kódban az egyes elnevezések (blokk, parancsblokk, vezérlőblokk, érték-blokk) az EBNF-nyelvtanhoz valamelyest hasonló, de hozzá képest önálló logikát követnek. A vizuális blokk-kód esetén a fő szempont az egyszerű érthetőség a precizitással (EBNF) szemben, azonban a kettőnek megfeleltethetőnek kell lennie egymásnak, hiszen csak így biztosítható, hogy a szöveges forráskód és a vizuális blokk-kód kölcsönösen megfeleltethetőek legyenek egymásnak.

Parancsblokkok:



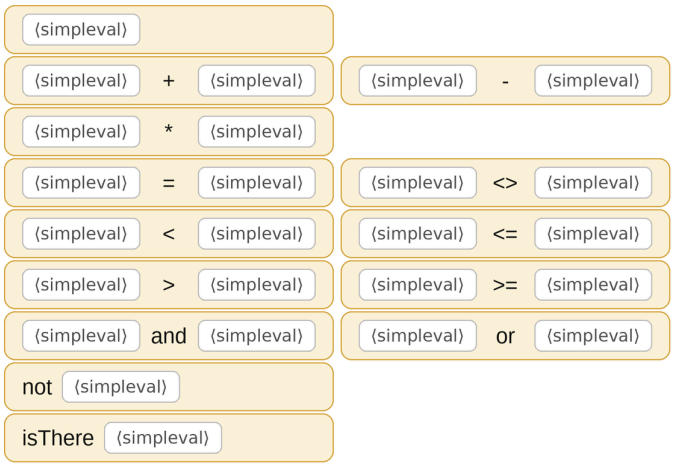
Vezérlő parancsblokkok:



Érték-blokkok:

Érték-blokkok a `set ⟨ident⟩ to ⟨calculatedval⟩` parancsblokk `to` utáni részének, vagyis az értékadás jobb oldalának a helyére illeszthetők. A `set` blokk az ide kerülő érték-blokkot értékeli ki, és az eredményt az `⟨ident⟩` nevű változóba írja. Az érték-blok tehát önmagában nem utasítás, mindig egy `set` blokk része, és nem ágyazható egy másik érték-blokk mezőjébe.

Az alábbi érték-blokkok használhatók:



Az egyes blokkok pontos működése - a kulcsszavaik alapján - a korábbi - nyelvtanról szóló - részből ismerhető meg.

A blokkok szerkeszthető mezőibe (a `⟨simpleval⟩`, `⟨ident⟩` helyére) a konkrét változóneveket, számokat vagy logikai értékeket kell megadni. Ezeket elegendő begépelni, nem elvárás, hogy listából is kiválaszthatók legyenek (pl. a már létező változók vagy a `true/false` értékek közül),

de ez is megvalósítható. A `<block>` helyére újabb parancsblokkok és vezérlő parancsblokkok sorozata kerülhet.

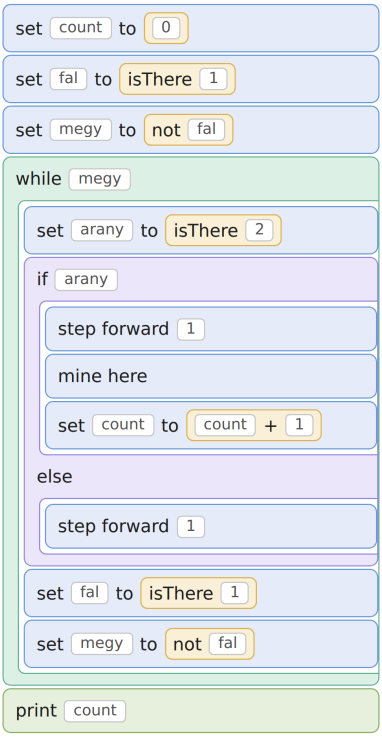
A vezérlő blokkok egymásba ágyazása legfeljebb három szint mély lehet. Az elfogadott maximális egymásba ágyazottságot mutatja ez a példa:

```
while
  if true
    if true
      print 1
```

Az `else` utasítás ahhoz a szinthez tartozik, amelyikhez a hozzá tartozó `if` tartozik.

A vizuális szerkesztőnek nem szabad több szintű programot engednie létrehoznia, azt pedig feltételezhetitek, hogy az input fájlban nem lesz ilyen programkód.

Íme a korábbi bonyolultabb program, ami vezeti a robotot előre, amíg falba nem ütközik. Út közben pedig minden aranyrögöt kibányász, majd a végén kiírja, mennyi aranyrögöt bányászott ki.



4. A naplózás

Tegyük fel, hogy az alkalmazásotok lefordított, futtatható fájljának neve `robot.exe`. Ha lefuttatjuk parancssorból az alábbi parancsot:

```
robot.exe input_001.txt output_001.txt
```

Akkor a napló állományt az IDE `output_001.txt` néven fogja létrehozni, és az minden újonnan elkezdett programfuttatáskor (függetlenül attól, hogy nyomkövetéssel vagy anélkül került elindításra a program) felülíródik.

A program futása közben az IDE minden feldolgozott programsor számát kiírja, olyan sorrendben, ahogy a sorok egymás után végrehajtásra kerülnek. Ciklusnál ez azt jelenti, hogy a törzs sorai minden lefutáskor újra megjelennek, a `while` sor száma pedig minden feltétel-kiértékelésnél, a visszaugrásnál is megjelenik. Az `else` sorának száma csak akkor kerül a naplóba, ha az `else` ág lefut, vagyis ha az `if` feltétele `false` volt.

Amennyiben a robot API-ja kerül meghívásra, akkor a sorszám után naplózásra kerül az is, hogy milyen API-végpont (a forráskódbeli alakjában, p`l. turn by, mine here),` milyen paraméterrel lett meghívva (futási idejű kiértékelt értékkel), és mi volt a visszatérési érték (ha van), ebben a sorrendben.
Ha **bármelyik sor** végrehajtása közben futási hiba keletkezik, a sor végére ki kell írni a hibakódot (a hibakódot mindig csupa nagybetűvel kell írni). **Egy sorban legfeljebb egy hiba keletkezhet, az, amelyik a sor kiértékelése során az első alkalommal jelentkezik.**

Ha egy API-hívás paramétere nem inicializált változó, a sorban az API hívás neve és az `E_UNINITIALIZED` hibakód szerepel, a paraméter nélkül (pl. `isThere E_UNINITIALIZED`).

A programfutás végét az `EOR` sor (end of run) jelzi (akkor is, ha hiba nélkül vagy hibakóddal fejeződött be a futás).

Pl. adott a következő kód:

```
set arany to isThere 2
set count to 0
turn by 0
print arany
if false
  print 1
else
  set x to isThere 3
print 9
```

Az `output_001.txt` naplófájlba a következő íródik a program elindítása után (feltételezve, hogy nincs arany a robot előtt).

```
1 isThere 2 false
2
3 turn by 0
4 print false
5
7
8 isThere 3 E_BAD_ARG
EOR
```

A naplófájl értelmezése:

- Minden sor elején a - szöveges forráskód reprezentáció szerinti - éppen futtatott sor száma áll: 1, 2, 3, ... Az utolsó sorban a programfutás vége jelzés (`EOR`) szerepel. (Üres program futtatása esetén a naplófájl egyetlen `EOR` sort tartalmazna.)
- Az 1. sor tartalmazza, hogy az `isThere` API került meghívásra egyetlen paraméter (`2`) átadásával, a visszatérési érték pedig `false` lett. **Mivel a nyelvtan szerint egy sorban csak egyetlen függvényhívás vagy parancs szerepelhet, ezért egy sorba se kerülhet egynél több API hívás naplózása.**
- A 2. sorban egy értékadás történt, így nincs extra naplózandó információ.
- A 3. sorban API került meghívásra (`turn by`) egyetlen paraméterrel, nincs visszatérési érték.
- A 4. sorban a `print` utasítás került meghívásra az API-n. A paraméter kiértékelt értéke került kiírásra.
- Az 5. sor egy `if`-et tartalmaz, a sor számán kívül nem kerül más kiírásra.
- (A 6-os sorszám hiányzik, mert az `if` nem teljesült.)
- A 7-tel **kezdődő sor** üres (`else` utasítás).
- A 8-as **kezdődő sorban** hiba történt, a futási hiba miatt kiírásra került a hibakód is.
- Egy soron belül az elemeket (a sorszámot, az API-végpont nevét, a paramétert, a visszatérési értéket és a hibakódot) pontosan egy szóköz választja el egymástól, a sorok végén pedig nincs felesleges szóköz.

4.1 Alternatív befejezés: futási limit

A program futása során legfeljebb 1000 programsor dolgozható fel. Ha a program az 1000. feldolgozott programsor után újabb sor feldolgozására készülne, a futás megszakad. Ilyenkor a naplófájl az 1000. feldolgozott programsor bejegyzése után egyetlen további sorként az `INSTRUCTION_COUNT_LIMIT_EXCEEDED` hibajelzést tartalmazza (ez a napló 1001. sora), az `EOR` sor pedig **nem kerül** a naplófájlba.

5. A robot szimulátor

A fejlesztői környezet egyik fő része az ún. robot szimulátor.

A szimulátor a térképet jeleníti meg, és rajta a robot aktuális helyzetét és irányát. Mutatja a már felfedezett mezőket – azokat, amelyeken a robot már járt, vagy amelyekre meghívódott az `isThere` függvény – megkülönböztetve a még feltérképezetlen területektől.

Emellett megjeleníti a robot érzékelőjének állapotát is: azt, hogy a robot aktuális helyzetéből és irányából nézve mi van közvetlenül előtte (üres kamra, fal vagy aranyrög), vagyis mit adna vissza ebben a pillanatban az `isThere` függvény. Az érzékelő állapota a robot minden mozgásakor és fordulásakor frissül, függetlenül attól, hogy a program meghívta-e az `isThere` függvényt. Ha a robot a térkép széle felé néz, az érzékelő falat mutat. Az érzékelő megjelenítése önmagában nem teszi felfedezetté a robot előtti mezőt.

A szimulátor azt is mutatja, hogy a robot eddig hány aranyrögöt bányászott ki. A már kibányászott aranyrögök helyét nem szükséges külön jelölni a térképen.

A szimuláció kétféleképpen történhet. Automatikusan, az éppen szerkesztett (vagy betöltött) program futtatásával, illetve kézzel, a robot API vezérlő végpontjainak (`turn by`, `step forward`, `mine here`) a felhasználói felületről történő, egyenkénti meghívásával. Kézi irányítás esetén az aktuális programfuttatás megszakad, és nem történik naplózás.

6. Beadandó

A csapatnak háromféle anyagot kell beadnia:

- a **programot**, annak forráskódját, futtatható változatát és a használt fejlesztői / build környezet leírását, valamint a mesterséges intelligencia használatának rövid leírását,
- egy **bemutató videót**: ami egyértelműen bizonyítja a program által megvalósított funkciók helyes működését a mellékelt forgatókönyv alapján,
- valamint a **verseny tesztek futtatásának eredményét**: a versenybizottság által, a csapat számára előkészített teszt input halmaz futásának az eredményét.

6.1 A beadandókkal kapcsolatos elvárások

A program. A teljes fejlesztői mappát adjátok be, ahogy az a gépeteken van: a teljes forráskódot és benne a lefordított, futtatható alkalmazást is, amennyiben az a programozási környezet alapján létezhet (pl. `robot.exe`). Ügyeljetek arra, hogy a lefordított alkalmazás ténylegesen futtatható legyen, és megfeleljen a forráskódnak. Készítsetek egy rövid `build.md` állományt, hogy milyen platformon / fejlesztői környezet alatt fordították a programot és hogyan.

Szintén a zip fájl gyökerébe, a `build.md` mellé helyezzetek el egy `ai.md` állományt, amelyben röviden leírjátok, hogyan használtatok mesterséges intelligenciát a fejlesztés során, ha használtatok. Az `ai.md` tartalmazza:

- milyen AI-eszközöket és modelleket használtatok (az eszköz és a modell neve, verziója);
- mire használtátok az AI-t, és nagyjából a program mekkora része készült AI segítségével;
- hogyan ellenőriztétek és javítottátok az AI által készített eredményt;
- 5 konkrét esetet, mindegyiket egy-két mondatban. Azokat az eseteket válasszátok, amelyek a legnagyobb hatással voltak a munkátok eredményére és minőségére, legyen szó akár sikeres, akár sikertelen AI-használatról.

Ha nem használtatok AI-t, az `ai.md`-ben ezt írjátok le, azt is leírva, miért döntöttetek így.

A bemutató videó. Egy legfeljebb 15 perces, mp4 formátumú felvétel[1], amely az alkalmazást működés közben mutatja be az alábbi forgatókönyv alapján, kitérve minden elkészült funkcióra. A felvételből **legfeljebb 10 perc lehet a programbemutató**, és **legfeljebb 5 perc egy kihívást jelentő programrész** bemutatása (lásd 1. sz. melléklet). A felhasználó felület úgy legyen alapból is megtervezve, hogy egyértelmű legyen a felhasználó számára, hogy a programot hogyan kell használni akkor is, ha a felhasználó nem ismeri még a programotokat. Fontos, hogy a videófelvételnek nem feltétlenül szükséges esztétikusnak lennie, a bemutató tartalma a fontos, tehát az, hogy a felvételen minden jól látszódjon: a felbontás legyen megfelelő (a feliratok olvashatóak, az esetleges grafikák jól láthatóak legyenek), az időzítés pedig legyen nyugodt (ne legyen kapkodó). Lehet beszéddel is segíteni a megértést, de ez nem kötelező. A versenybizottság a videót az értékelés során többször megnézi és meg-megállítja, de ha a helyes működést aránytalanul nagy időráfordítással, csak aprólékos, sokszori ismételt visszatekeréssel lehet megérteni, vagy az nem egyértelmű, akkor az érintett feladatrészre a csapat 0 pontot kap.

A bemutató videó forgatókönyvét az 1. mellékletben találhatjátok.

A verseny tesztek futtatásának eredménye. A versenyhétvége végén a bizottság bemeneti teszteseteire (`input_001.txt`, `input_002.txt`, …) le kell futtatnotok az alkalmazásotokat, és a keletkező kimeneteket egyetlen `results.zip` fájlban beküldenetek. A kimeneti fájlok neve pontosan a bemenetekhez igazodjon: `output_001.txt`, `output_002.txt`, és így tovább. A sorszámozott input fájlok kihagyás nélkül követik egymást.

6.2 Mit hová kell feltölteni

A programot, a videót és a verseny tesztek futtatásának eredményeit SFTP[2]-n, a csapat saját mappájába kell feltölteni a Discordon megadott SFTP-szerverre.

A csapat SFTP-mappájának szerkezete:

```
/csapatnev/
├── input.7z      ← a bizottság teszi ide; jelszóval védett
├── project.zip   ← FELTÖLTÉS vasárnap 19:00-ig (program forráskód és bináris + build.md
                    + ai.md)
└── bemutato.mp4 ← FELTÖLTÉS vasárnap 19:00-ig (a bemutató videó)
└── results.zip   ← FELTÖLTÉS vasárnap 19:30-ig (verseny teszt futtatás eredmények)
```

Az `input.7z`-et a bizottság helyezi el a csapat mappájában; ez tartalmazza a verseny tesztek bemeneti fájljait. A fájl 7z tömörítővel tömörített és jelszóval védett, kitömöríteni csak a vasárnap 19:00-kor Discordon közölt jelszóval tudjátok.

A `project.zip` fájlt a csapat tölti fel, és tartalmazza az alkalmazásotok forráskódját és annak lefordított változatát (bináris), valamint a `build.md` és az `ai.md` állományt a zip fájl gyökerében.

[1] Videó felvételéhez használhatjátok az OBS Studio nevű ingyenes alkalmazást.
[2] SFTP kliensnek Windowson használható a WinScp alkalmazás. Linuxon és MacOS-en elérhető az sftp parancs.

A `bemutato.mp4` fájl az alkalmazás felhasználói felületének működéséről készített videó. Max. 500MB hosszúságú lehet.

A keletkező kimeneteket egyetlen `results.zip`-be csomagolvaí 19:30-ig töltsétek fel. A `results.zip` a kimeneti fájlokat a zip gyökerében tartalmazza (nem almappában), pontosan a bemenetekhez igazodó néven (`output_001.txt`, `output_002.txt`, …).

6.3 A verseny tesztek futtatása és időrend

Az SFTP-szerverre **vasárnap 19:00-ig várjuk a programot és a videót. Az ez után az időpont után feltöltött eme fájlokat már nem tudjuk figyelembe venni.**

19:00 és 20:00 között a csapatoknak folyamatosan rendelkezésre kell állniuk, mert ekkor fog zajlani a verseny tesztek futtatása és eredményeinek beküldése, amelyhez a csapatok közreműködése szükséges.

Időpont	**Teendő**
A hétvége során	Az `input.7z` (jelszóval védett) letöltése az SFTP-mappából
Vasárnap 19:00-ig	A `project.zip` és `bemutato.mp4` feltöltése SFTP-n
Vasárnap 19:00	A jelszó közlése Discordon; az `input.7z` kitömörítése és a program lefuttatása minden bemenetre
Vasárnap 19:30-ig	A `results.zip` feltöltése SFTP-n. **Ügyeljetek arra, hogy a sok tesztfájl (200+) futtatása miatt beférjetek a fél órás időkeretbe.** Ehhez készüljetek fel arra, hogy a nagyszámú tesztesetet automatikusan - egymás után - könnyen le tudjátok futtatni. (Ezt segíti elő a feladat elején kért `-q` paraméter, azaz a csendes futtatás használata.)

7. Értékelési szempontok

Az elkészített alkalmazás konzolos alkalmazás vagy grafikus felülettel rendelkező (Graphical User Interface-es, azaz GUI-s) alkalmazás is lehet. **Pontozás szempontjából egyik sem jelent előnyt a másikhoz képest a verseny 1. fordulójában.**

A versenybizottság a beadott videó és program alapján ellenőrzi, hogy az alkalmazás a felhasználói felületén helyesen megvalósítja-e az elkészült funkciókat.

Az IDE értékelésénél a feladatban kért funkciók helyes működésén túl a felület áttekinthetőségét is figyelembe vesszük: mennyire könnyen érthető és használható a felület annak is, aki most találkozik vele először. Emellett értékeljük a feladat által nem kért, de ötletes, a használatot segítő fejlesztéseket is, ha azok a bemutató videóban szerepelnek.

A forráskód és a beküldött binárison keresztül kerül majd ellenőrzésre, hogy a videó és a beküldött verseny teszt eredmények valóban a csapat programjának az eredményei.

A verseny tesztek segítségével kerül teljeskörűen tesztelésre a programfuttatás és a robotszimulátor logikájának helyes megvalósítása. Kisméretűek lesznek a tesztállományok, így nem szükséges arra felkészülnötök, hogy nagyméretű inputot hatékonyan kezeljetek.

A fejlesztés során használhattok mesterséges intelligenciát (AI-t). A feladat szövegének megértéséhez, kérdések feltételéhez a versenybizottság kifejezetten javasolja ennek használatát! A programozást tekintve viszont azt javasoljuk, hogy a generált programkód tüzetes ellenőrzésével és/vagy előre definiált teszteseteken keresztül ellenőrizzétek a programkódot, így garantálható a szoftver helyessége és az értékelésnél a magas pontszám!

Az AI használata önmagában nem befolyásolja a pontszámot: sem több, sem kevesebb pont nem jár azért, ha egy csapat használt AI-t, vagy ha nem használt. Azt viszont értékeljük, hogy aki használta, mennyire tudatosan tette: mire használta, és hogyan ellenőrizte, javította a kapott eredményt. Ezt az `ai.md` alapján ítéljük meg.

Pontozandó elem	**Maximális pontérték**
Programfuttatás helyes működése (verseny tesztek)	160 pont
Felhasználói felület működése (videó + beküldött program alapján), `build.md` és `ai.md`	140 pont
Összesen	**300 pont**

Ha a program (project.zip) nem kerül határidőn belül megfelelően feltöltésre, akkor az összpontszám 0 pont lesz. Ha a bemutato.mp4 nem kerül feltöltésre, akkor a felhasználói felület működése 0 pont lesz.

A 2. fordulóban a továbbjutó csapatok az 1. fordulóban készített alkalmazást fejleszthetik tovább (akár újra implementálással is).

Jó munkát kíván a versenybizottság!

1. sz. melléklet: A bemutató videó forgatókönyve

A videó a felhasználói felületet mutatja be az alábbi bemeneten és az alábbi lépések sorrendjében. A nyelvi szabályokat, a hibakódokat és a naplóformátumot a verseny tesztek ellenőrzik, ezeket nem kell külön részletesen bemutatni. Ha egy funkció nem készült el, vagy valami miatt nem tudjátok pontosan a forgatókönyv szerint bemutatni az alkalmazást, azt jelezzétek felirattal vagy hangalámondással a videó közben vagy a végén. Ha nem folytatható egy az egyben a bemutató a forgatókönyv szerint, próbáljátok áthidalni a hiányosságot, hogy minél több mindent bemutathassatok az alkalmazásotokból, miközben - lehetőség szerint - minél inkább közel maradtok az eredeti forgatókönyvhöz és annak szándékához.

Ha a felvétel közben elrontotok valamit, vagy a forgatókönyvtől eltérő lépést tesztek, nem kell megállnotok és elölről kezdenetek a felvételt. Javítsátok ki a hibát a felületen, majd folytassátok onnan a forgatókönyvet. Ez teljesen rendben van, sőt, így a program több működése is látszik. Ügyeljetek arra, hogy a programbemutató (a forgatókönyv 1–9. pontja) ezzel együtt is beleférjen a 10 perces időkeretbe.

Bemenet (`input_video.txt`)

A fájl csak a térképet tartalmazza, **a program üres**:

```
5 10
.#..@..#.@
.#.##.##..
>.@.@@.@.#
##.#..#.#.
.@..#..@..
```

A vizuális szerkesztőben az alábbi programot kell majd megszerkesztenek a lenti lépések sorrendjében:

```
set count to 0
set megy to true
while megy
  set arany to isThere 2
  if arany
    step forward 1
    mine here
    set count to count + 1
  else
    step forward 1
  set fal to isThere 1
  set megy to not fal
print count
```

A videó menete

1. **Indítás.** Indítsátok az alkalmazást parancssorból:
   `robot.exe input_video.txt output_video.txt`
2. **Áttekintés.** Mutassátok meg a térképet a robottal és a szenzort.
3. **Szerkesztés.** Állítsátok elő a fenti programot üres állapotból a blokkszerkesztővel az alábbi sorrendben, közben a szöveges nézetnek is követnie kell a változásokat. Az első lépéseknél még nem létező változókat használtok, a felületnek ezeket el kell fogadnia.
   3.1. `while megy`
   3.2. `print count` a `while` mögé
   3.3. `if-else` blokk `arany` feltétellel a `while` törzsébe
   3.4. `step forward 1` az `if` és az `else` ágba is
   3.5. `mine here` az `if` ágban a `step forward 1` mögé
   3.6. `set count to count + 1` a `mine here` mögé
   3.7. `set arany to isThere 2` az `if-else` elé
   3.8. `set fal to isThere 1` az `if-else` mögé, ennek mögé `set megy to not fal`
   3.9. `set megy to true` a `while` elé, ennek elé `set count to 0`
4. **Futtatás nyomkövetés nélkül.** Látszódjon a robot végállapota, a felfedezett terület, a szenzor és a konzol. Nyissátok meg a naplófájlt.
5. **Futtatás nyomkövetés nélkül a programkód betöltésével.** Lépjetek ki az alkalmazásból. Az `input_video.txt` állományba vigyétek fel a programkódot, majd így indítsátok el a futtatást nyomkövetés nélkül. Látszódjon a robot végállapota, a felfedezett terület, a szenzor és a konzol. Nyissátok meg a naplófájlt.
6. **Nyomkövetés.** Indítsátok újra a programot. Lépegessetek végig a programon - mutatva a változók tartalmát - amíg az le nem áll. Közben több alkalommal is - a program futásának végén is - nyissátok meg a naplófájlt.
7. **Kézi vezérlés.** Indítsátok újra a programot. A nyomkövetés közben hagyjátok a robotot, amíg az első aranyrögig el nem jut. Kézzel bányásszátok ki az aranyrögöt, és irányítsátok el a robotot az összes aranyröghöz, amíg mindegyik aranyrögöt ki nem bányásztátok. A robot menjen falnak is időnként, ütközésnél hiba nélkül álljon meg.
8. **Ideiglenes módosítások.** Minden módosítás után futtassatok, mutassátok meg az eredményt, és csak utána töröljétek a beszúrt blokkot.
   8.1. Próbáljatok egy `set` blokk változónevének `while`-t, majd `9lives`-t megadni.
   8.2. Szúrjatok be az `if arany` ág végére egy `if true` blokkot, abba `print 1`-et. Próbáljatok az `if true` törzsébe még egy vezérlőblokkot tenni, ezt nem szabad engedni (max. 3 szintű struktúra). Futtatás után töröljétek az `if true` blokkot a tartalmával együtt.
   8.3. Szúrjatok be a program végére egy `mine here` blokkot. Futtatás közben hibaüzenetet kell látnunk. Ezután töröljétek a blokkot.
   8.4. Szúrjatok be a program végére egy `while true` blokkot, a törzsébe pedig egy `print 1` blokkot. Futtatás közben a programnak magától le kell állnia, és a felületen látszódnia kell az utasításszám-limit túllépését jelző hibaüzenetnek (`INSTRUCTION_COUNT_LIMIT_EXCEEDED`). Ezután töröljétek a `while true` blokkot a tartalmával együtt.

9. **Egyéb fontos funkciók ismertetése.** Ha van olyan működés, amit fontosnak tartotok bemutatni és a fenti forgatókönyvben nem jön elő, akkor azt illesszétek be a forgatókönyv végére! Ezeket a fejlesztéseket is értékeljük.
10. **Kihívást okozó programrészlet bemutatása**. Mutassátok be a program egy olyan részét, amelynek a megvalósítása kihívást jelentett számotokra, akár sikerült végül megoldani, akár nem. Térjetek ki arra, hogy mi okozta a nehézséget, milyen megoldási módokkal próbálkoztatok, és mit tanultatok belőle. Ezt a részt hangalámondással kísérjétek, mert a magyarázat nélkül nem követhető. **Ez a rész legfeljebb 5 perc lehet, és nem számít bele a programbemutató 10 perces időkeretébe.**